Department of Computer Science and Engineering
University of Moratuwa

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing

Software Requirements Specification

Version <2.0>

Project ID(PID)	11
Group Number	25
Mentor	Mr. Bhanuka Siriwardhana

Group Members

Registration No.	Name
230667F	Virusan T.
230670H	Vishula J.
230674A	R.P.M.Vithanage

Revision History

Date	Version	Description	Author
15/07/2026	0.1	Project commenced - stakeholder analysis and initial requirement elicitation	Group 25
22/07/2026	0.5	Draft functional requirements for all five backend modules and web frontend	Group 25
29/07/2026	0.9	Initial SRS draft - functional and non functional requirements for auth and user, marketplace, ingestion and ETL, Admin, Notification, and React web frontend	Group 25
09/08/2026	1.0	Initial SRS release - requirements reviewed and aligned with SAD v1.0	Group 25
10/08/2026	2.0	Corrected SAD/SRS sections that contradicted the reduced-team decision to keep the synchronous ETL→Marketplace write boundary (rejecting an event-driven refactor). Fixed: ADR-002, section 3.1 data isolation bullet, section 3.4.3 IAM table, section 9 Data View, section 11.1 Reliability, section 3.10 (SRS) , all previously stated "zero exceptions" / event-based writes, now correctly document the single scoped MarketplaceVehiclesWriteAdapter exception. Removed stray event-driven language from SRS FR-14, FR-18.1, FR-43. Updated team size/timeline (3→2 developers, 6 weeks remaining) in SAD section 3.2 and SRS section2.4.	Group 25

Table of Contents

Group Members 1
1. Introduction 7
1.1 Purpose 7
1.2 Scope 7
1.3 Definitions, Acronyms, and Abbreviations 9
1.4 References 10
1.5 Overview 11
2. Overall Description 11
2.1 Product Perspective 11
2.2 Product Functions 12
2.3 User Characteristics 12
2.4 Constraints 13
2.5 Assumptions and Dependencies 13
3. Specific Requirements 14
3.1 Functionality 14
3.1.1 Auth & User Service 14
3.1.2 Marketplace Service 15
3.1.3 Ingestion & ETL Service 17
3.1.5 Notification Service 19
3.1.6 Web Frontend (by user role) 20
3.2 Usability 20
3.3 Reliability 20
3.4 Performance and Security 21
3.5 Supportability 22
3.6 Design Constraints 22
3.7 On-line User Documentation and Help System Requirements 23
3.8 Purchased Components 23
3.9 Interfaces 23
3.9.1 User Interfaces 23
3.9.2 Hardware Interfaces 24
3.9.3 Software Interfaces 24
3.9.4 Communications Interfaces 24
3.10 Database Requirements 24
3.11 Licensing, Legal, Copyright, and Other Notices 25
3.12 Applicable Standards 25
4. Supporting Information 26
Appendix A: Data Dictionary 26
Appendix B: Sample CSV/JSON Upload Schema 35
B.1 CSV Format 35
B.2 JSON Format (equivalent) 35
B.3 ZIP Image Archive 36

Individual Contributions

Index No.	Name	Contribution
230667F	Virusan T.	• Section 1.2 Scope • Section 2.1 Product perspective • Section 2.2 Product functions • Section 3.1.2 Marketplace - FR-13 to FR-15 • Section 3.1.2 Marketplace - FR-16 to FR-18 • Section 3.1.2 Marketplace - FR-19 to FR-21.4 • Section 3.1.2 Marketplace - FR-22 to FR-25 • Section 3.1.6 Web Frontend - FR-54 to FR-55 • Section 3.2 Usability - NFR-01 to NFR-04 • Section 3.4 Performance - NFR-09 to NFR-12.1 • Section 3.7 On-line user documentation and help system requirements • Section 3.9.1 User interfaces • Appendix A - marketplace.vehicles, vehicle_images, favourites • Appendix A - marketplace.search_queries, vehicle_dictionaries • SRS ↔ SAD requirements traceability review
230670H	Vishula J.	• Section 1.3 Definitions & glossary • Section 2.4 Constraints • Section 2.5 Assumptions and dependencies • Section 3.1.3 Ingestion & ETL - FR-26 to FR-31 • Section 3.1.3 Ingestion & ETL - FR-30.1 to FR-30.2 • Section 3.1.3 Ingestion & ETL - FR-32 • Section 3.1.3 Ingestion & ETL - FR-33 to FR-33.7 • Section 3.1.3 Ingestion & ETL - FR-34 to FR-35.2 • Section 3.1.3 Ingestion &

		ETL - FR-36 to FR-38 • Section 3.1.3 Ingestion & ETL - FR-39 to FR-41.3 • Section 3.1.3 Ingestion & ETL - FR-42 to FR-42.1 • Section 3.1.5 Notification - FR-49 to FR-53 • Section 3.1.6 Web Frontend - FR-56 to FR-58 • Section 3.3 Reliability - NFR-05 to NFR-08 • Section 3.5 Supportability - NFR-21 to NFR-26.1 • Section 3.6 Design constraints - NFR-27 to NFR-33 • Section 3.8 Purchased components • Section 3.9.3 Software interfaces • Section 3.9.4 Communications interfaces • Appendix A - ingestion.upload_jobs, rejected_records, etl_stage_logs • Appendix A - notification.notifications • Appendix B - Sample CSV format & clean/messy example rows • Appendix B - Sample JSON format & specs preservation note • Appendix B - ZIP image archive naming convention (FR-35)
230674A	Vithanage R.P.M.	• Section 1.1 Purpose • Section 1.4 References • Section 1.5 Overview • Section 2.3 User characteristics • Section 2 Modular monolith clarification sentence (v1) • Section 3.1.1 Auth & User - FR-01 to FR-08 • Section 3.1.1 Auth & User - FR-09 to FR-12 • Section 3.1.4 Admin - FR-43 to FR-48 • Section 3.1.6 Web Frontend - FR-59 • Section 3.4 Security -

		NFR-13 to NFR-20 • Section 3.9.2 Hardware interfaces • Section 3.10 Database requirements • Section 3.10 Modular monolith clarification sentence (v1) • Section 3.11 Licensing, legal, copyright, and other notices • Section 3.12 Applicable standards • Section 4 Supporting information & cross-references • Appendix A - auth.users, auth.dealer_profiles, auth.refresh_tokens • Appendix A - admin.audit_logs • Revision history • Table of contents & IEEE 830 structure compliance

1. Introduction

This Software Requirements Specification (SRS) specifies software requirements for Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing. The document contains a detailed description of functional and non-functional software requirements, system constraints, interfaces and quality attributes of the system. It is intended to be the main source of information for developers, testers, project supervisors and other interested parties involved in the software development process.

The proposed system is a web application designed for the ease of managing inventories of second-hand vehicles by dealers as well as for providing intelligent searching capability for buyers.It contains an **automated, Step Functions-orchestrated ETL pipeline** which will automatically process uploaded vehicle information and images, validate records, normalize data, generate semantic search embeddings and save processed data into PostgreSQL database.

The SRS is prepared based on the IEEE 830 standard of Software Requirements Specification and is structured in accordance with the Rational Unified Process (RUP) SRS template.

1.1 Purpose

This Software Requirement Specification (SRS) has been developed to establish the entire set of software requirements for the Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing System. This document provides the detailed description of the system behavior, functions, non-functional requirements, constraints on design, and external interfaces.

The objectives of this SRS are to:

- Provide a clear understanding of the software system to all project stakeholders.
- Define the functional requirements that describe how the system should operate.
- Specify non-functional requirements relevant to the system.
- Serve as the primary reference for software design, implementation, testing, deployment and maintenance.
- Establish a common agreement between developers, supervisors, testers and clients regarding the scope and expected behaviour of the software.
- Reduce ambiguity during software development by documenting all software requirements before implementation.

This document will be used throughout the project lifecycle as the baseline for system development and verification.

1.2 Scope

The “Second Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing” web application on the cloud allows vehicle dealers to handle and publish their vehicle inventories, as well as provides customers with a way to search and browse through vehicles.

The main goal of the system is to automate the inventory processing process for vehicle dealers, who can now upload their entire inventory using CSV files along with ZIP files consisting of images of their vehicles instead of entering each listing individually.

The ETL pipeline performs the following operations:

- Slicing large inventory files into manageable chunks for parallel processing.
- Validation of uploaded vehicle records with row-level rejection capabilities.
- Data normalization to maintain consistent information, utilizing a deterministic rule-based parser with a fallback to the Groq AI API for complex contextual normalization.
- Image extraction, resizing, compression and optimization via Sharp.
- Generation of semantic vector embeddings for intelligent search using a locally hosted all-MiniLM-L6-v2 model.
- Storage of processed vehicle information in PostgreSQL, utilizing JSONB attributes to handle polymorphic vehicle specifications (e.g., varying fields for CAR, BIKE, VAN).
- Notification of upload completion, including aggregate success and rejection statistics, through email.

The solution also offers intelligent marketplaces for buyers through the combination of filter-based search and semantic search. Buyers have the ability to look up vehicles either using filters or through natural language query search. The search engine follows a dual strategy: assigning confidence scores to parse deterministically and employing Groq AI for low confidence queries before running a PostgreSQL search using pgvector and pg_trgm.

The application supports three primary user roles:

- Buyer – Searches vehicles, views vehicle details and manages favourite listings.
- Dealer – Uploads inventories, manages vehicle listings, monitors upload jobs and receives notifications.
- Administrator – Monitors system activities, manages users, reviews upload jobs, maintains audit logs and oversees platform operations.

The system architecture leverages a modern, cloud-native stack to ensure scalability and performance:

- Frontend: React
- Backend: NestJS
- Database: PostgreSQL with the pgvector extension
- Cloud Infrastructure: Amazon S3 (file storage), Amazon SQS (asynchronous processing), and Amazon SES (email notifications)
- DevOps: Docker (containerization), Terraform (infrastructure provisioning) and GitHub Actions (CI/CD)

The following features are outside the scope of this project:

- Online vehicle payments.
- Vehicle financing services.
- Insurance management.
- Legal ownership transfer.
- Real-time chat between buyers and dealers.

These features may be considered in future system enhancements

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ETL	Extract, Transform, Load - the automated pipeline that ingests, validates, and stores dealer-uploaded vehicle data.
JWT	JSON Web Token - a signed token used for stateless authentication.
RBAC	Role-Based Access Control - authorization model restricting actions by user role (Guest, Buyer, Dealer, Admin).
S3	Amazon Simple Storage Service - object storage used for uploaded files and vehicle images.
SQS	Amazon Simple Queue Service - managed message queue used to decouple upload requests from ETL processing.
SES	Amazon Simple Email Service - used to send upload status notification emails.
RDS	Amazon Relational Database Service - managed hosting for the PostgreSQL database.
pgvector	PostgreSQL extension enabling storage and similarity search over vector embeddings.
pg_trgm	PostgreSQL extension enabling trigram-based fuzzy text matching.
NL search	Natural-language search - free-text queries such as "automatic SUV under 20 million LKR".

Term	Definition
DLQ	Dead-letter queue - where messages are routed after repeated processing failures.
Lambda	AWS Lambda - the serverless compute service that executes each module's containerized handlers.
ECR	Amazon Elastic Container Registry - stores the Docker images deployed to Lambda.
IAM	AWS Identity and Access Management - controls per-service permissions.
CI/CD	Continuous Integration / Continuous Deployment - automated build, test, and deployment pipeline.
IaC	Infrastructure as Code - AWS infrastructure defined and provisioned via Terraform.
UAT	User Acceptance Testing - end-of-cycle testing performed from the user's point of view.
MVP	Minimum Viable Product - the first deployable version covering core scope only.

1.4 References

[1] AutoTrader, "AutoTrader – Buy & Sell Cars." Available: https://www.autotrader.com/

[2] Cars.com, "Cars for Sale – New and Used Cars." Available: https://www.cars.com/

[3] ikman.lk, "Vehicles for Sale in Sri Lanka." Available: https://ikman.lk/

[4] Riyasewana, "Vehicles for Sale in Sri Lanka." Available: https://riyasewana.com/

[5] NestJS, "NestJS Documentation." Available: https://docs.nestjs.com/

[6] PostgreSQL Global Development Group, "PostgreSQL Documentation." Available: https://www.postgresql.org/docs/

[7] pgvector, "Vector Similarity Search for PostgreSQL." Available: https://github.com/pgvector/pgvector

[8] Amazon Web Services, "AWS Documentation." Available: https://docs.aws.amazon.com/

[9] React, "React Documentation." Available: https://react.dev/

[10] Docker, "Docker Documentation." Available: https://docs.docker.com/

[11] HashiCorp, "Terraform Documentation." Available: https://developer.hashicorp.com/terraform/docs

[12] GitHub, "GitHub Actions Documentation." Available: https://docs.github.com/actions

[13] Hugging Face, "Sentence-Transformers: all-MiniLM-L6-v2." Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[14] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

[15] Group 25 (PID 11), Project Proposal - Second-Hand Vehicle Marketplace, University of Moratuwa, 2026.

[16] Group 25 (PID 11), Feasibility Study - Second-Hand Vehicle Marketplace, University of Moratuwa, 2026.

1.5 Overview

The Software Requirements Specification (SRS) document is structured to have four broad sections. After the introductory section, the Section 2 provides an overview of the system by describing its product perspective, major functions, characteristics of users, design constraints, assumptions, and dependencies. In Section 3, the functional and non-functional requirements for every backend module and the web frontend are provided along with interface requirements, database requirements, design constraints, and standards.

2. Overall Description

2.1 Product Perspective

The system is a new, standalone cloud-native product; it does not replace or integrate with an existing legacy system. It consists of a React frontend, an AWS API Gateway entry point, and five independently deployable backend services implemented in NestJS/TypeScript and run as Docker based AWS Lambda functions:

- Auth & User Service : registration, login, JWT authentication, roles and dealer profile approval.
- Marketplace Service: vehicle listings, favourites, dealer data and hybrid search engine
- Ingestion & ETL Service: bulk upload of vehicle details utilizing an AWS Step Functions state machine triggered by SQS.
- Admin Service : user management, upload monitoring, reports, audit logs and dashboard queries.
- Notification Service : event driven email notifications via Amazon SES.

These services share one Amazon RDS PostgreSQL instance (with pgvector and pg_trgm extensions) using service-owned schemas. All AWS infrastructure is provisioned through Terraform and deployed via GitHub Actions/Amazon ECR.

2.2 Product Functions

- Dealer registration, login and profile/approval management.
- Bulk inventory upload via CSV/JSON plus ZIP image archives.
- Automated ETL pipeline: validation, normalization, image processing, embedding generation
- Upload job status tracking and per-record error reporting for dealers.
- Filter-based and natural-language (hybrid semantic) vehicle search for registered buyers and guests.
- Buyer favourites and vehicle detail browsing.
- Automated email notification of upload completion or failure.
- Administrative monitoring of users, uploads, audit logs and platform reports.
- Synthetic vehicle data generation tool to support testing without needing real dealer data.

2.3 User Characteristics

Role	**Characteristics**
Guest	Unauthenticated visitor; no technical background assumed; can browse and search but cannot save favourites or upload.

Role	Characteristics
Buyer	General public or a B2B professional familiar with vehicle specifications and comfortable with traditional searching, filtering and saving capabilities
Dealer	Semi-technical business user, who is comfortable working with inventory files in CSV or JSON format and managing image ZIP files, but not expected to have programming knowledge.
Administrator	Platform operator with higher technical familiarity; expected to understand user management, audit logs and basic system health indicators.

2.4 Constraints

- Development budget is limited to AWS Free Tier usage; no paid licenses are used.
- The project is delivered by a team of three undergraduate developers within a fixed academic timeline (26 June – 3 October).
- Backend technology is fixed to NestJS/TypeScript on AWS Lambda; frontend is fixed to React; database is fixed to PostgreSQL.
- The backend architecture must operate within AWS Lambda's hard guardrails, specifically the 15-minute maximum execution timeout per function.
- The hybrid search engine's and ETL pipeline's reliance on Groq AI is subject to the provider's free-tier rate limits (Requests Per Minute and Tokens Per Minute), requiring mechanisms if throttling occurs during peak search loads.
- Payments, financing, insurance and real-time chat are excluded from this version's scope.
- All infrastructure must be defined as Terraform code rather than created manually in the AWS console.

2.5 Assumptions and Dependencies

Assumptions

- Dealers have their vehicle inventory available, or can be made available, in CSV or JSON format with an accompanying ZIP of images.
- An AWS account with Free Tier eligibility is available to the team for the duration of the project.
- The Hugging Face all-MiniLM-L6-v2 model remains available and license-compatible for bundling inside the Marketplace and ETL services.

- It is assumed that the Groq AI free tier request and token limits will be sufficient to handle both the hybrid search engine's load and the ETL pipeline's normalization tasks without requiring a paid license.
- GitHub and GitHub Actions remain available for version control and CI/CD throughout the project.
- Users access the system over a standard broadband or mobile internet connection using a modern web browser.

Dependencies

- The software relies on external AWS infrastructure (Lambda, RDS, SQS, SES, Step Functions, API Gateway, S3) to function properly.
- Step Functions Payload Constraints: The chunked ETL process depends on the raw data payloads staying under AWS limits.
- The system is dependent on Groq AI for query parsing in the search engine and for row-level validation in the ETL pipeline.

3. Specific Requirements

3.1 Functionality

Functional requirements are organized by module/subsystem, matching the system's deployment architecture, followed by the web frontend requirements organized by user role.

3.1.1 Auth & User Service

- FR-01: The system shall allow a new user to register as a Buyer or Dealer, providing email, password, and role based profile fields.
- FR-02: New Dealer registrations shall require the Dealer to declare a dealer type of either Individual or Business. All new registrations shall be placed in a PENDING verification state and require Administrator approval before the account is activated.
- FR-02.1: Dealers registering as Business shall be required to provide a valid business registration number and upload supporting verification documents, such as a business registration certificate. Dealers registering as Individual shall not be required to provide a business registration number.
- FR-02.2: The system shall record the identity of the Administrator who approved or rejected each Dealer's verification request, together with the date and time of the approval or rejection decision.
- FR-03: The system shall authenticate users via email/password and issue a JWT access token and refresh token on successful login.

- FR-04: The system shall allow a valid refresh token to be exchanged for a new access token without re-authentication.
- FR-05: The system shall support refresh token revocation on logout or password change.
- FR-06: The system shall enforce a minimum password strength policy (e.g., minimum 8 characters, requiring mixed case and numbers) at registration and password change.
- FR-07: The system shall store only salted password hashes, never plaintext passwords.
- FR-08: The system shall assign each user exactly one role (Buyer, Dealer or Administrator) and embed role claims in the JWT.
- FR-09: The system shall allow Administrators to approve or reject pending Dealer registrations, triggering an automated email notification of the decision
- FR-10: The system shall allow authenticated users to view and update their own profile.
- FR-11: The system shall allow Administrators to deactivate a user account without deleting its historical data.
- FR-12: The system shall not allow public registration for the Administrator role; Administrator accounts shall be provisioned internally via database seeding or an authenticated Administrator.

3.1.2 Marketplace Service

- FR-13: The system shall allow Dealers to create, update and deactivate individual vehicle listings.
- FR-13.1: When an edit to a listing changes any field contributing to its searchable text, the system shall regenerate that listing's search_text and its embedding vector before the listing returns to a publicly visible state.
- FR-13.2: A listing whose searchable text has changed shall not remain searchable using its previous embedding. Failing to regenerate the vector would leave the listing described by its former content and therefore unfindable by the terms it now contains.
- FR-14: The Ingestion & ETL pipeline's Load stage shall perform bulk inserts of validated listings directly into marketplace.vehicles, using a column-scoped write grant confined to a single adapter class. This is a deliberate, narrow exception to service write ownership (see ADR-002); the Marketplace Service does not separately consume or process this data.
- FR-15: Each listing shall store core relational data as relational columns and shall store category-specific specifications in a JSONB specs column. This is necessary because required attributes vary by vehicle type.
- FR-16: The system shall allow authenticated Buyers to save and remove vehicles from a personal favourites list.
- FR-17: The system shall allow Buyers to retrieve their favourites list on demand.
- FR-18: The system shall expose dealer profile summaries alongside listings.

- FR-18.1: The Marketplace Service shall retrieve dealer profile summaries via a scoped, read-only cross-schema query against auth.dealer_profiles (per the least-privilege role model in section3/ADR-001), rather than maintaining a separately synchronized copy of dealer data.
- FR-19: The system shall allow any user to search listings using structured filters (make, model, price range, mileage range, fuel type, transmission).
- FR-20: The system shall return paginated, sortable search results.
- FR-21: The system shall parse natural-language queries into structured filters using a deterministic rule-based parser utilizing dictionary lookups and pg_trgm pattern matching.
- FR-21.1: The system shall calculate a Confidence Score based on the ratio of consumed meaningful tokens versus unparsed tokens.
- FR-21.2: If the Confidence Score falls below a predefined threshold (e.g., 0.6), the system shall route the original query and partial JSON to the Groq AI API to extract the remaining structured filters.
- FR-21.3: The system shall enforce a validation layer on AI-generated outputs to drop hallucinated values before merging them into the final structured query.
- FR-21.4 :The system shall log every unresolved token encountered during the processing of natural-language queries, along with any corrections applied by the Groq AI fallback. Corrections that recur with sufficient frequency shall be promoted to the vehicle-dictionary vocabulary. This will allow subsequent occurrences of the same correction to be resolved through the deterministic rule-based parser without requiring an additional Groq AI API call.
- FR-22: The system shall generate a query embedding for semantic matching against listing embeddings.
- FR-22.1: The embedding model used to generate listing vectors during ingestion and the embedding model used to generate query vectors during search shall be identical in model, version, pooling strategy, and normalization. Both shall be invoked through a single shared library rather than configured independently per service.
- FR-22.2: Any change to the embedding model shall require regenerating the embeddings of all existing listings. Partial migration is not permitted: vectors produced by different models occupy incompatible coordinate spaces and yield confidently incorrect similarity results rather than an error, so a mismatch would degrade search quality without producing any detectable failure.
- FR-23: The system shall combine hard SQL filters (via WHERE clauses) and semantic similarity ranking (via ORDER BY embedding ⇔ query_vector) into a single, optimized PostgreSQL query.
- FR-24: The system shall degrade gracefully to rule-based filtering and trigram search if the external Groq AI API fails (e.g., timeout, rate-limit), or bypass semantic ranking entirely if a search query resolves fully to structured filters leaving no leftover semantic text
- FR-25: The system shall display related/recommended listings on a vehicle's detail page.

3.1.3 Ingestion & ETL Service

- FR-26: The system shall accept a Dealer's CSV/JSON inventory file with an accompanying ZIP of vehicle images.
- FR-27: The system shall validate file type and size before accepting an upload.
- FR-28: The system shall store each raw uploaded file in Amazon S3 exactly as received, without modification.
- FR-28.1: Stored raw files shall not be altered or overwritten by any subsequent pipeline stage. They serve as the authoritative record of what the Dealer submitted, enabling the pipeline to be re-run against the original input after a defect is corrected, and enabling disputes about submitted values to be resolved against the source.
- FR-29: The system shall create an upload job record for each submitted inventory file and route the event to AWS Step Functions via SQS.
- FR-30: The system shall utilize a Lambda function to split large upload batches into manageable chunks.
- FR-30.1: The AWS Step Functions state machine shall utilize a Map State to process these data chunks in parallel across multiple concurrent Lambda invocations.
- FR-30.2: Each Step Functions stage shall write its output payload to Amazon S3 and pass only a reference to that object to the next stage. No stage shall pass row data directly through Step Functions state.
- FR-31: The system shall return a job identifier to the dealer immediately upon upload submission.
- FR-32: The system shall allow Dealers to poll upload job status, including progress, validation errors and summary statistics.
- FR-33: The system shall normalize inconsistent input values using a rule-based parser and shall calculate a per-field confidence score for every required field, followed by a single row-level confidence score
- FR-33.1: Each field's confidence score shall be determined by the method that resolved it
- FR-33.2: The row-level confidence score shall be the minimum of its required fields' scores, never an average. A single unresolved required field shall therefore reduce the entire row's score to that field's value, so that one corrupted field cannot be masked by other correctly-resolved fields.
- FR-33.3: The description field shall be excluded from confidence scoring entirely. It is unstructured input that is never expected to resolve to a structured value, and shall neither lower nor raise a row's score.
- FR-33.4: Rows with a Confidence Score below 0.6 shall be routed to a dedicated Groq AI Lambda for LLM-based context normalization.

- FR-33.5: The Groq AI normalization request shall contain only the flagged row's unresolved raw values, the row's description field, and a fixed instruction block specifying the allowed enumerated values for each closed-vocabulary field. Rows that meet the confidence threshold shall not be included in any prompt.
- FR-33.6: Where a field's vocabulary is large (make and model), the request shall include a shortlist of candidate values rather than the complete vocabulary. The shortlist exists to constrain the model's output to strings that will match a stored database value, not to enumerate the domain for the model.
- FR-33.7: The system shall not include already-resolved rows, unrelated columns or the full contents of any dictionary table in a Groq AI request.
- FR-34: The system shall validate required fields and numeric ranges per record, rejecting invalid records without failing the whole batch.
- FR-35: The system shall match uploaded images to the correct listing using the vehicle’s registration number, normalized for case and formatting before comparison.
- FR-35.1: Rows whose registration number duplicates another row within the same upload shall be rejected to the rejected records table with an explicit reason, since a duplicate key makes automated image matching unreliable.
- FR-35.2: Rows with a blank registration number shall not be rejected - unregistered vehicles are a legitimate case - but shall be accepted without an automated image match and flagged for the dealer to resolve manually during review.
- FR-36: The system shall generate a search_text aggregate for each listing by concatenating its essential relational fields, its JSONB specification values and unmapped column keys, and its full description text. The search_text value shall then be embedded using the bundled all-MiniLM-L6-v2 model to produce a 384-dimensional vector, stored on the listing prior to database insertion.
- FR-36.1: The description field shall be included in search_text in full and unmodified. Free-text detail supplied by the Dealer that could not be resolved into any structured field remains meaningful to semantic search, and is frequently the phrasing a Buyer uses when searching.
- FR-36.2: Only search_text shall be embedded. The system shall not embed the raw row object
- FR-37: The system shall resize, compress and convert vehicle images to a consistent format.
- FR-38: The system shall reject corrupted or unsupported image files with an error attributed to the specific listing row.
- FR-39: The system shall aggregate the results of the Map State before passing the payload to the Notification Service.
- FR-40: The system shall process ETL messages idempotently, avoiding duplicate listings on reprocessing.

- FR-41: The system shall retry transient processing failures with exponential backoff, and shall degrade to reduced-accuracy processing rather than failing an upload when a non-transient failure occurs.
- FR-41.1:When a Groq AI call is rate-limited, the system shall retry and if it times out, returns an error, or returns malformed JSON, the system shall proceed with the deterministic rule-based normalization results for the affected rows rather than failing the chunk.
- FR-41.2: A row that remains incomplete after both the rule-based and AI normalization attempts shall be written to the rejected-records table with a stated reason, and shall not prevent the remaining rows in its chunk from completing
- FR-41.3: The system shall route repeatedly-failing messages to a dead-letter queue.
- FR-42: Listing produced by the ETL pipeline shall be persisted with PENDING REVIEW status. No ETL-loaded listing shall become publicly visible until the owning Dealer explicitly approves it.
- FR-42.1: The Dealer's reviewing interface shall indicate which fields on a PENDING REVIEW listing were inferred by the Groq AI normalisation step, along with the model's stated reasoning for that inference, and shall present rows in ascending order of normalization confidence score so the least certain rows are reviewed first.

3.1.4 Admin Service

- FR-43: The Admin Service shall call the Auth Service synchronously via the internal API Gateway(e.g.,POST/internal/dealers/{id}/approve,.../reject,POST internal/users/{id}/deactivate) to apply account state changes. The Auth Service performs the write in its own schema in response; the Admin Service separately writes the corresponding audit record to admin.audit_logs (see ADR-005, section3.5.2).
- FR-44: The system shall allow Administrators to view all upload jobs across all dealers.
- FR-45: The system shall generate summary reports (listing counts, upload volumes, error rates, active users) over a selectable date range.
- FR-46: The system shall record sensitive administrative actions in an audit log with actor identity and timestamp.
- FR-47: The system shall allow audit logs to be searched and reviewed.
- FR-48: The system shall expose aggregated key metrics for the Administrator dashboard.

3.1.5 Notification Service

- FR-49: The system shall listen for upload-completed and upload-failed events published.
- FR-50: The system shall render a templated summary email for each completed or failed upload.
- FR-51: The system shall send notification emails via Amazon SES.

- FR-52: The system shall record the delivery status of each notification.
- FR-53: The system shall retry sending on transient SES failures without duplicate-sending a notification that already succeeded.

3.1.6 Web Frontend (by user role)

- FR-54: The system shall allow Guests to browse the catalogue and use filter/NL search without registering.
- FR-55: The system shall prompt an unauthenticated Guest to register or log in when attempting to save a favourite.
- FR-56: The system shall provide Dealers a drag-and-drop interface for bulk upload of CSV/JSON and ZIP files.
- FR-57: The system shall display polled upload job progress and row-level error reports to Dealers.
- FR-58: The system shall allow Dealers to manage (edit/deactivate) their own listings via the frontend.
- FR-59: The system shall provide Administrators a dashboard for user management, upload monitoring, and audit log review.

3.2 Usability

- NFR-01: A new Dealer shall complete their first successful bulk upload within 15 minutes of registration, unaided.
- NFR-02: Search/filter UI patterns shall follow conventions familiar from existing regional marketplaces.
- NFR-03: Validation error messages shall be in plain language and reference the specific row/field of the uploaded file.
- NFR-04: The web frontend shall be usable on both desktop and tablet-width viewports.

3.3 Reliability

- NFR-05: The system shall target nearly 99% availability during active development/demonstration periods.
- NFR-06: ETL job failures shall be logged with sufficient detail for diagnosis and be retryable without data loss or duplication.
- NFR-07: Validation shall catch out-of-range numeric values before persistence; rejected records shall be reported, not silently discarded.
- NFR-08: Defects shall be classified critical/significant/minor, with critical defects blocking release.

3.4 Performance and Security

Performance:

- NFR-09: The system will have an average API response time of under 500 ms for all CRUD and browse APIs. The average response time to natural language search queries should be below 2 seconds for warm start AWS Lambda invocations. However, the response time to cold start AWS Lambda invocations, which need to provision a container image of about 3008 MB to host the embeddings model and its inferencing engine, may take more time than the threshold set.

- NFR-10: The system shall support at least 50 concurrent buyer sessions and 5 concurrent dealer bulk uploads without degraded response times. The system must utilize an adequately sized database connection pool or Amazon RDS Proxy to safely handle the resulting 50 simultaneous database write connections.

- NFR-11: The system shall be designed to support up to 10,000 listings and 100 concurrent users at prototype scale.

- NFR-12: AWS Lambda memory and timeouts shall be tuned individually per function. Heavy dependency functions (embed function and the search Lambda) shall be allocated ~3008 MB. The image processing function (images function) shall be allocated ~2048 MB. All other HTTP-facing and rule-based Lambdas shall be allocated between 256 MB and 512 MB to minimize cold-start latency.

- NFR-12.1: The search and ingestion parsers shall degrade gracefully to rule-based filtering and parsing if the external Groq LLM API times out, rate-limits, or returns malformed JSON. Additionally, if a search query resolves entirely to structured filters leaving no leftover semantic text, the system shall skip vector embedding and order results by default scalar criteria (e.g., price or date). .

Security:

- NFR-13: Authentication shall be JWT-based and stateless, with short-lived access tokens and revocable refresh tokens.

- NFR-14: Authorization shall be enforced via role-based guards at the controller level.

- NFR-15: Passwords shall be hashed and never stored or logged in plaintext.

- NFR-16: All client-backend traffic shall be served over HTTPS.

- NFR-17: Each Lambda function shall be granted least-privilege IAM permissions.

- NFR-18: All API request bodies shall be validated and sanitized to prevent injection attacks.

- NFR-19: Vehicle images shall not be publicly writable; access shall be via signed URLs

- NFR-20: Secrets shall be stored in AWS Secrets Manager or SSM Parameter Store, never hard-coded.

3.5 Supportability

- NFR-21: All code shall conform to TypeScript, ESLint, and Prettier standards enforced via CI.
- NFR-22: Every service shall follow the shared NestJS module structure
- NFR-23: Each service shall maintain unit/integration/end to end test directories with an agreed minimum coverage target.
- NFR-24: Logs shall be structured (JSON) and centralized in Amazon CloudWatch.
- NFR-25: Each service repository shall include a README covering setup, environment variables and deployment.
- NFR-26: Code changes shall go through feature branches and reviewed pull requests.
- NFR-26.1: The normalization dictionaries and embedding-generation logic shall be maintained as a single shared library consumed by both the Marketplace and Ingestion services.

3.6 Design Constraints

- NFR-27: Backend services shall be implemented in NestJS/TypeScript and deployed as AWS Lambda functions. Lightweight functions shall be packaged as ZIP archives; functions with large dependencies shall be packaged as Docker container images.
- NFR-28: PostgreSQL (RDS) with pgvector and pg_trgm shall be the only relational data store used.
- NFR-29: The frontend shall be implemented in React.
- NFR-30: All AWS infrastructure shall be defined as Terraform code.
- NFR-31: Development and deployment shall remain within AWS Free Tier usage limits.
- NFR-32: The Ingestion & ETL pipeline shall be orchestrated via AWS Step Functions state machine rather than a single monolithic Lambda worker, so that large uploads are processed as many short-lived, independently retryable Lambda invocations. This provides declarative per-stage retry policies, automatic fan-in after parallel chunk processing, and independent memory sizing per stage.
- NFR-33**:** The system shall utilize the Groq AI API for dynamic text normalization; the system must handle rate limits and potential API timeouts gracefully without crashing the ETL pipeline.

3.7 On-line User Documentation and Help System Requirements

- In-app contextual guidance is provided on the Dealer upload screen, describing the expected CSV/JSON column layout and offering a downloadable sample template.
- A static FAQ/help page covers account registration, using search and filters and troubleshooting failed uploads.
- A dedicated live-chat help desk is not planned for this version; each backend repository's README serves as developer-facing documentation.

3.8 Purchased Components

The following components, which have been commercially purchased or licensed, do not fall within the scope of this project. The following third party components are open source or used in the typical pay per use manner under the AWS Free Tier: React, NestJS, PostgreSQL, pgvector, Sharp, Hugging Face all-MiniLM-L6-v2 model, Terraform, Docker, and AWS services mentioned in this document. Further, the system utilizes the Groq API for LLM inference as part of the natural language parsing process in the free/metered tier form.

3.9 Interfaces

3.9.1 User Interfaces

The following screens are required. Wireframes are not finalized at this stage; this section lists the functional elements each screen must contain.

- Guest/Buyer browse & search page : filter panel (make, model, price range, mileage range, fuel type, transmission etc), a natural-language search box, sort control and a paginated grid of listing cards.
- Vehicle detail page : image gallery, full specification list, dealer contact summary and a save-to-favourites action (prompting login if the user is a guest).
- Buyer favourites page : list of saved vehicles with the ability to remove an item and jump to its detail page.
- Login / register page : role selection (Buyer or Dealer), credential fields, and password-policy hints.
- Dealer upload page : drag-and-drop area for the CSV/JSON file and ZIP image archive, a template download link, and an upload-submit action.
- Dealer job status / error report page : progress indicator, batch-level breakdown and a filterable table of row-level validation errors.
- Dealer listings management page : searchable table of the dealer's own listings with edit and deactivate actions.
- Administrator dashboard : key metric tiles (total listings, pending approvals, jobs in progress, recent failures).

- Administrator user management page : table of dealer/buyer accounts with approve/suspend/reactivate actions.
- Administrator audit log page : searchable, timestamped log of sensitive administrative actions.

3.9.2 Hardware Interfaces

None. The system requires only a client device capable of running a modern web browser; no direct hardware interfacing is required.

3.9.3 Software Interfaces

- AWS API Gateway REST interface between the React frontend and all backend Lambda functions.
- AWS SDK integration for S3 (file storage), SQS (asynchronous messaging), and SES (email) from the relevant services.
- PostgreSQL client/ORM interface (with pgvector and pg_trgm extensions) shared by the Marketplace and Ingestion & ETL services.
- Hugging Face sentence-transformers (all-MiniLM-L6-v2) runtime bundled inside the Marketplace and ETL services for embedding generation.
- GitHub Actions, Docker and Amazon ECR interfaces used for building, publishing, and deploying container images.

3.9.4 Communications Interfaces

- Synchronous HTTPS/REST requests between the frontend and API Gateway, and between API Gateway and each backend service.
- Asynchronous messaging between the Ingest API and the Step Functions state machine via Amazon SQS.
- Asynchronous event delivery from the AWS State machine to the Notification Service and outbound email delivery via Amazon SES.
- Dealer-facing job status updates use polling against the Job Status API; no WebSocket or push-based channel is planned for this version.

3.10 Database Requirements

Five PostgreSQL schemas are used for the system architecture, and a separate schema is assigned to each backend application: auth, marketplace, ingestion, notification, and admin. Each backend has its own role with the least privilege that allows full read-write access to only its corresponding schema.

Cross-schema read access is allowed only in cases where there is already a foreign key dependency between them, such as marketplace.vehicles.dealer_id referencing auth.users.id, giving the marketplace application read access to auth.users table.

Cross-schema writes are not permitted, with exactly one narrow, database-enforced exception: the Ingestion & ETL pipeline's Load stage performs INSERT/UPDATE operations into marketplace.vehicles under a column-scoped grant, confined in code to a single adapter class (MarketplaceVehiclesWriteAdapter), because it and Marketplace's listings table have a strict 1:1 producer–consumer relationship (see ADR-002, section5.2.3). No other backend application performs INSERT/UPDATE/DELETE operations on another application's schema. When an administrative action needs to modify data belonging to another service, the modification is performed through that service's API (see ADR-005) rather than a direct write.

The Admin Service has read-only access to all other schemas to support cross-cutting reporting and dashboards. It does not perform direct writes outside its own admin schema. When an administrative action needs to modify data belonging to another service, the modification is performed through that service's API.

The auth schema includes the users, dealer_profiles, and refresh_tokens tables. The marketplace schema includes the vehicles, vehicle_images, favorites, search_queries, and vehicle_dictionaries tables. The ingestion schema includes the upload_jobs, rejected_records, and etl_stage_logs tables. The notifications table belongs to the notifications schema, and the audit_logs table belongs to the admin schema.

The vehicles table stores universal data on the vehicles such as make, model, year, price, mileage, dealer, and status as relational columns. Specifications of the type-specific and variable data, such as body_type, number of seats, and other specific for each vehicle type attributes, are stored as the specs JSONB column. It is discussed in more details in FR-15. The vehicle_dictionaries table provides a possibility of using a controlled vocabulary in order to correct dealer and buyer provided make and model texts via fuzzy matching using the PostgreSQL trigram search.

The vehicle listings embeddings are stored in the fixed-size vector(384) column due to the fact that the all-MiniLM-L6-v2 produces embeddings with the dimension of 384. The database backups are made automatically using Amazon RDS.

3.11 Licensing, Legal, Copyright, and Other Notices

- Dealers must accept a terms-of-service statement confirming that uploaded vehicle details and images are accurate and that they hold the right to publish them; the system does not knowingly store third-party copyrighted images without permission.
- User passwords and personal data are hashed/encrypted at rest and handled in line with the principles of Sri Lanka's Personal Data Protection Act, No. 9 of 2022, as an academic best-practice reference.
- Open-source license notices for all bundled third-party components are retained in each repository's LICENSE/NOTICE files.

3.12 Applicable Standards

- IEEE Std 830-1998 - structure and content of this SRS.
- OWASP Top 10 - baseline web application security considerations.

- WCAG 2.1 Level AA - aspirational accessibility target for the web frontend.
- REST architectural conventions for all HTTP API design.
- ISO 8601 - date and time formatting across all services.
- Semantic Versioning - version numbering for each independently deployable service.

4. Supporting Information

Table of Contents of this document is automatically generated at the start of the document. Appendices of this document of SRS are not available. Data Dictionary and sample schema for CSV/JSON file upload will be added as appendices to next version of SRS after finalisation of schema.

This SRS should be read in conjunction with the Project Proposal, Feasibility Study and Sprint Planning Document of the project.

Appendix

Second-Hand Vehicle Marketplace with Intelligent Search and Automated Inventory Processing - SRS Appendix

This appendix fulfils the placeholder in Section 4 (Supporting Information) of the SRS. It is intended to be inserted at the end of the SRS document, after Section 4, as Appendix A and Appendix B. It reflects the five-schema database design and functional requirements (FR-15, FR-35, FR-36, Section 3.10) as of this SRS revision.

Appendix A: Data Dictionary

The database is organized as five PostgreSQL schemas, one per backend service, per Section 3.10. Each table below is listed under its owning schema. Primary keys are marked PK; foreign keys are marked FK with their target.

auth.users

Column	Type	Constraints / Keys	Description
id	uuid	PK, default gen_random_uuid()	Unique user identifier
email	varchar(320)	Unique (case-insensitive)	Login identifier
password_hash	varchar(255)	NOT NULL	Salted password hash (FR-07); never plaintext
name	varchar(255)	NOT NULL	Display name
role	enum(buyer, dealer, admin)	NOT NULL, default buyer	Single role embedded in JWT claims (FR-08)

Column	Type	Constraints / Keys	Description
is_active	boolean	NOT NULL, default true	False if deactivated by an Administrator (FR-11)
created_at	timestamptz	NOT NULL, default now()	Account creation timestamp
updated_at	timestamptz	NOT NULL, default now()	Last profile update timestamp

auth.dealer_profiles

One-to-one with a Dealer user. Business-only fields are nullable so Individual dealers are never forced to carry empty business data.

Column	Type	Constraints / Keys	Description
user_id	uuid	PK, FK → auth.users.id	Owning user
dealer_type	enum(individual, business)	NOT NULL	Declared at registration (FR-02); determines required documents
business_registration_number	varchar(100)	nullable	Required for Business dealers only (FR-02.1)
business_address	varchar(255)	nullable	Required for Business dealers only
verification_documents	jsonb	NOT NULL, default {}	S3 key references - NIC (Individual) or registration certificate (Business)
verification_status	enum(pending, verified, rejected)	NOT NULL, default pending	Approval state gating account activation (FR-02)
verified_by	uuid	nullable, FK → auth.users.id	Administrator who decided (FR-02.2)
verified_at	timestamptz	nullable	Decision timestamp (FR-02.2)
contact_number	varchar(50)	nullable	Dealer contact number
created_at	timestamptz	NOT NULL, default now()	

auth.refresh_tokens

Column	Type	Constraints / Keys	Description
id	uuid	PK	Token record identifier
user_id	uuid	FK → auth.users.id	Owning session
token_hash	varchar(255)	NOT NULL, unique	Hash of the refresh token; raw token never stored
expires_at	timestamptz	NOT NULL	Expiry
revoked_at	timestamptz	nullable	Set on logout or password change (FR-05)
created_at	timestamptz	NOT NULL, default now()	

marketplace.vehicles

Universal fields are relational columns; type-specific attributes live in specs (FR-15).

Column	Type	Constraints / Keys	Description
id	uuid	PK	Listing identifier
dealer_id	uuid	FK → auth.users.id	Owning dealer
upload_job_id	uuid	nullable, FK → ingestion.upload_jobs.id	Null for manually created listings
vehicle_type	enum(car, motorcycle, van, truck, other)	NOT NULL	
make / model	varchar(100)	NOT NULL	Corrected against vehicle_dictionaries where needed
condition, fuel_type, transmission_type	enum	condition NOT NULL; others nullable	Closed-vocabulary attributes
manufacture_year / registration_year	smallint	manufacture_year NOT NULL	
price	numeric(12,2)	NOT NULL	
is_negotiable	boolean	NOT NULL, default false	

Column	Type	Constraints / Keys	Description
mileage	integer	nullable	
registration_number	varchar(50)	nullable, unique	Image-matching key, case/format normalized (FR-35)
chassis_number	varchar(100)	nullable, unique	
description	text	nullable	Dealer free text; included verbatim in search_text (FR-36.1)
specs	jsonb	NOT NULL, default {}	Category-specific attributes (FR-15)
search_text	text	nullable	Generated aggregate embedded to produce the vector (FR-36)
embedding	vector(384)	nullable	From the shared embedding library (FR-22.1); regenerated on qualifying edits (FR-13.1)
search_vector	tsvector	nullable	Full-text index derived from search_text
status	enum(draft, pending_review, active, sold, archived, rejected)	NOT NULL, default draft	ETL-loaded listings start at pending_review (FR-42)
created_at / updated_at	timestamptz	NOT NULL, default now()	updated_at refreshed whenever search_text/embedding regenerate

marketplace.vehicle_images

Column	Type	Constraints / Keys	Description
id	uuid	PK	
vehicle_id	uuid	FK → marketplace.vehicles.id	
s3_path	varchar(1024)	NOT NULL	Original stored image
processed_path / thumbnail_path	varchar(1024)	nullable	Sharp-optimized outputs

Column	Type	Constraints / Keys	Description
is_primary	boolean	NOT NULL, default false	Cover image for listing cards
display_order	integer	NOT NULL, default 0	Gallery ordering

marketplace.favourites

Column	Type	Constraints / Keys	Description
id	uuid	PK	
buyer_id	uuid	FK → auth.users.id	
vehicle_id	uuid	FK → marketplace.vehicles.id	
created_at	timestamptz	NOT NULL, default now()	

marketplace.search_queries

user_id is nullable specifically so Guest searches can still be logged for analytics.

Column	Type	Constraints / Keys	Description
id	uuid	PK	
user_id	uuid	nullable, FK → auth.users.id	Null for Guest searches
raw_text	text	NOT NULL	Original free-text query
extracted_filters	jsonb	NOT NULL, default {}	Parsed structured filters
confidence_score	numeric(3,2)	nullable	Per FR-21.1
used_groq_fallback	boolean	NOT NULL, default false	Whether FR-21.2 was triggered
search_time	timestamptz	NOT NULL, default now()	
result_count	integer	NOT NULL, default 0	

marketplace.vehicle_dictionaries

Backs trigram-based make/model correction (Section 3.10) and the alias-promotion loop in FR-21.4.

Column	Type	Constraints / Keys	Description
id	uuid	PK	
parent_id	uuid	nullable, FK → marketplace.vehicle_dictionaries.id	Links a model to its parent make
dictionary_type	enum(make, model, fuel_type, transmission_type)	NOT NULL	
canonical_value	varchar(150)	NOT NULL	The correct, stored value
aliases	jsonb	NOT NULL, default []	Corrections promoted from recurring Groq fallback fixes (FR-21.4)
created_at	timestamptz	NOT NULL, default now()	

ingestion.upload_jobs

Column	Type	Constraints / Keys	Description
id	uuid	PK	Returned to the dealer immediately (FR-31)
dealer_id	uuid	FK → auth.users.id	
file_name	varchar(255)	NOT NULL	
csv_s3_path	varchar(1024)	NOT NULL	Raw file, unmodified (FR-28)
zip_s3_path	varchar(1024)	nullable	
status	enum(pending, processing, completed, failed, cancelled)	NOT NULL, default pending	
total_records / valid_records / invalid_records	integer	NOT NULL, default 0	Polled by the dealer (FR-32)

Column	Type	Constraints / Keys	Description
created_at / updated_at	timestamptz	NOT NULL, default now()	

ingestion.rejected_records

Column	Type	Constraints / Keys	Description
id	uuid	PK	
upload_job_id	uuid	FK → ingestion.upload_jobs.id	
row_number	integer	NOT NULL	Row position in the original file
raw_data	jsonb	NOT NULL	Original row values, for dealer review
reason	varchar(1000)	NOT NULL	e.g. duplicate registration number (FR-35.1), incomplete after AI fallback (FR-42.2)
created_at	timestamptz	NOT NULL, default now()	

ingestion.etl_stage_logs

Column	Type	Constraints / Keys	Description
id	uuid	PK	
upload_job_id	uuid	FK → ingestion.upload_jobs.id	
stage	enum(validate, normalize, enrich, embed, image_processing,load)	NOT NULL	
status	enum(pending, processing, completed, failed, skipped)	NOT NULL, default pending	
retry_count	integer	NOT NULL, default 0	Per FR-42's exponential backoff

Column	Type	Constraints / Keys	Description
started_at / completed_at	timestamptz	nullable	
error_message	text	nullable	Diagnostic detail (NFR-06)

notification.notifications

Column	Type	Constraints / Keys	Description
id	uuid	PK	
user_id	uuid	FK → auth.users.id	
type	enum(general, upload_completed, upload_failed, dealer_approved, dealer_rejected)	NOT NULL	
subject / message	varchar(255) / text	NOT NULL	Rendered from a template (FR-51)
payload	jsonb	NOT NULL, default {}	Structured context (e.g. job summary counts)
status	enum(pending, sent, failed)	NOT NULL, default pending	Delivery status (FR-53)
sent_at	timestamptz	nullable	
created_at	timestamptz	NOT NULL, default now()	

admin.audit_logs

Column	Type	Constraints / Keys	Description
id	uuid	PK	
actor_id	uuid	nullable, FK → auth.users.id	Administrator who performed the action
action	varchar(255)	NOT NULL	e.g. "dealer.approved", "user.deactivated"

Column	Type	Constraints / Keys	Description
entity_type	varchar(100)	NOT NULL	e.g. "dealer_profile", "user"
entity_id	uuid	nullable	Affected record
created_at	timestamptz	NOT NULL, default now()	

Appendix B: Sample CSV/JSON Upload Schema

It is the format in which the Dealer uploads (FR-26). This format is very similar to the format in which a dealer would upload the file after exporting from the spreadsheet, prior to any kind of normalization. The second row of data below is intentionally messy to show the kind of input this parser and AI would be processing.

B.1 CSV Format

Required column headers, in order:

vehicle_type,make,model,body_type,condition,manufacture_year,registration_year,price,is_negotiable,mileage,fuel_type,transmission_type,registration_number,chassis_number,color,engine_capacity_cc,owners_count,location_city,location_district,description

Sample rows:

```
car,Toyota,Corolla,sedan,used,2018,2018,8500000,false,45000,petrol,automatic,CAB-1234,MR2ABC1234567,White,1800,1,Colombo,Colombo,"Well maintained, full service history"

car,Toyata,Corrola,,used,2018,2018,8.5m,false,95k,deisel,auto,CAB-5678,MR2XYZ7654321,Silver,1800,2,Kandy,Kandy,"1.5L turbo petrol hybrid, full option AC"
```

Row 1 is correct and parsed deterministically. Row 2 showcases some examples of spelling mistakes ("Toyata"/"Corrola"), shortened measurements ("8.5m", "95k"), and a type of fuel ("deisel") which the rule-based parser normalizes using the fuzzy dictionary match (Section 3.10) and without using Groq AI - following the example of normalization from FR-33.

B.2 JSON Format (equivalent)

```
[
  {
    "vehicle_type": "car", "make": "Toyota", "model": "Corolla",
    "body_type": "sedan", "condition": "used", "manufacture_year": 2018,
    "price": 8500000, "mileage": 45000, "fuel_type": "petrol",
    "transmission_type": "automatic", "registration_number": "CAB-1234",
    "description": "Well maintained, full service history"
  }
]
```

Unrecognized JSON keys, and any CSV columns beyond the required set, are preserved into the listing's specs JSONB column rather than rejected, consistent with FR-15.

B.3 ZIP Image Archive

Images are matched to rows by registration_number, normalized for case and formatting before comparison (FR-35). Multiple photos of the same vehicle are distinguished by a numeric suffix; the first (unsuffixed or "-1") image is treated as primary.

images.zip

```
├── CAB-1234.jpg        (primary image for CAB-1234)
├── CAB-1234-2.jpg      (additional image, same vehicle)
├── CAB-5678.jpg
└── CAB-5678-2.jpg
```

Rows where the value in registration_number is NULL are permitted (unregistered cars being a valid example, per FR-35.2) but do not have an image match by the system, and are marked to be handled by the dealer when reviewing the rows on their screen. Rows where the value in registration_number is the same as that in another row in the upload are disallowed explicitly for a reason (FR-35.1).